

Functional Programming in Python

Based on Functional Programming with Python
Why It's Good To Be Lazy?

Concepts discussed

- Functional objects
- Closure
- Map, filter, reduce
- List comprehension
- List generator
- Lazy evaluations
- Curried functions
- Type inferencing
- Polymorphism
- Higher-order functions
- Referential Transparency

What about the real world?

- ❑ Functional programming is not mainstream
- ❑ But it widens your perspective on programming
- ❑ Pure functional programming is difficult
- ❑ Languages borrow concepts from the functional world
- ❑ Recent revival due to a need for concurrency

Functional Python

- Python is **not** a functional language
- But has some functional features...

First-class functions/functional objects

Lambda defines an anonymous function

```
def square(x):  
    return x**2
```

equivalent to

```
square = lambda x: x**2
```

Closures

Closure is a function with bound variables, or a function that retains a record of its environment at creation

```
def myfunc(n):  
    return lambda a : a * n
```

```
mydoubler = myfunc(2)    //n is 2  
print(mydoubler(11))    //a is 11
```

```
mytripler = myfunc(3)    //n is 3  
print(mytripler(11))    //a is 11
```

Closure, mydoubler and mytripler, can be seen as a “**functional object**”

Closures

```
def build_taxer(rate):  
    def taxer(amount):  
        return amount * (float(rate) / 100)  
    return taxer
```

```
taxer_CA = build_taxer(7.25)  
tax = print(taxer_CA(1000))  
taxer_ND = build_taxer(5)  
tax = print(taxer_ND(1000))
```

Map, filter and reduce (Python 3)

High-order functions operating on lists (sequences)

- Apply a function to every element

```
list(map(lambda x: x**2, range(1,5)))  
-> [1, 4, 9, 16]
```

- Select elements matching the predicate

```
list(filter(lambda x: x%2 == 0, range(10)))  
-> [0, 2, 4, 6, 8]
```

- Cumulatively reduce elements to a single value

```
import functools  
functools.reduce(lambda x,y: x+y, range(1,5))  
-> 10
```

Why map and reduce are so useful?

- Can simplify complex loops
- Can be chained
- Many computations can be reduced to those (not only numeric ones)
- Can be easily distributed (see Google's MapReduce)

Imperative primes, **the first** approach

```
def is_prime(n):  
    k = 2  
    while k < n:  
        if n % k == 0:  
            return False  
        k += 1  
    return True
```

- List of statements to execute one after another
- Not obvious if and when the loop ends

Primes, the **second** approach (High order function)

```
def is_prime(n):  
    return len(list(filter(lambda k: n%k!=0, range(2,n)))) == 0  
  
print(is_prime(20))
```

- Clear intention: “Is the list of non-trivial divisors empty?”
- High-order functions can be composed
- No side effects

List comprehensions

Creating a list based on existing lists.

- *List comprehensions* borrowed from Haskell

```
[i**2 for i in range(1,10) if i%2==0]  
-> [4, 16, 36, 64]
```

- Inspired by mathematical notation (slight difference)
 $\{i^2 \mid i \in \mathbf{N}, i \in [1, 10) : i \equiv 0 \pmod{2}\}$
- Can replace `map` and `filter` (even `lambda`)
- Simplifies complex chains (more dimensions)

Primes, the **third** approach (list comprehension)

```
def is_prime(n):  
    return True not in [True for k in range(2,n) if n%k == 0]
```

```
print(is_prime(20))
```

- Is there a problem?
- Do we have to go through the whole list?

Generators and iterators

It is said that good programmers are lazy...

- *Iterators* are lazy sequences
- *Generator expressions* help building iterators

```
(i**2 for i in range(1,10) if i%2==0)  
-> <generator object at 0x12c4850>
```

- `Map` and `filter` are lazy in Python 3

Prime numbers, the **fourth** approach (iterator and generator)

```
def is_prime(n):  
    return True not in (n%k==0 for k in range(2,n))
```

```
is_prime(20)  
-> False
```

- Lazy evaluation

Prime numbers, **fourth** approach (tracing)

```
def divisible(n, k):  
    if(n % k == 0):  
        print(k)  
        return True  
    else:  
        return False  
  
def is_prime(n):  
    return True not in (divisible(n, k) for k in range(2,n))  
  
is_prime(1000000)
```

Quantification

Can do even better!

- `any(seq)` returns true if at least one element of the sequence is true ($\exists$, *exists*)
- `all(seq)` returns true if all elements of the sequence are true ($\forall$ *for all*)

Short-circuit lazy evaluation, like with logical operators

Primes, grand finale

```
def is_prime(n):  
    return (not any(n%k==0 for k in range(2,n)))
```

An example

Imagine an accounting routine used in a book shop. It works on a list with sublists, which look like this:

Order Number	Book Title and Author	Quantity	Price per Item	
34587	Learning Python, Mark Lutz	4	40.95	
98762	Programming Python, Mark Lutz	5	56.80	
77226	Head First Python, Paul Barry	3	32.95	
88112	Einführung in Python3, Bernd Klein	3	24.99	

Write a Python program, which returns a list with 3-tuples. Each tuple consists of an product's order number, its order total, and a description of 'met the goal ' if the product's order total is equal or higher that the set-goal, or 'failed the goal' otherwise.

Implementation from the tutorial:

#assume minimum order is 100

```
orders = [  
    ["34587", "Learning Python, Mark Lutz", 4, 40.95],  
    ["98762", "Programming Python, Mark Lutz", 5, 56.80],  
    ["77226", "Head First Python, Paul Barry", 3, 32.95],  
    ["88112", "Einführung in Python3, Bernd Klein", 3, 24.99]  
]
```

```
invoice_totals = list(map(lambda x:  
    [x[0], x[1], "met the goal"] if x[1] >= 100  
    else [x[0], x[1], "failed the goal"],  
    map(lambda x: [x[0], x[2] * x[3]], orders)))  
  
print(invoice_totals)
```

Lambda and map

```
orders = [  
    ["34587", "Learning Python, Mark Lutz", 4, 40.95],  
    ["98762", "Programming Python, Mark Lutz", 5, 56.80],  
    ["77226", "Head First Python, Paul Barry", 3, 32.95],  
    ["88112", "Einführung in Python3, Bernd Klein", 3, 24.99]  
]
```

```
def fun1(min_order, seq):  
    return map(lambda x:  
        [x[0], x[1], "met the goal"] if x[1] >= min_order  
        else [x[0], x[1], "failed the goal"],  
        map(lambda x: [x[0], x[2] * x[3]], seq))  
  
print(list((fun1(200, orders))))
```

List comprehension

```
orders = [  
    ["34587", "Learning Python, Mark Lutz", 4, 40.95],  
    ["98762", "Programming Python, Mark Lutz", 5, 56.80],  
    ["77226", "Head First Python, Paul Barry", 3, 32.95],  
    ["88112", "Einführung in Python3, Bernd Klein", 3, 24.99]  
]
```

```
def fun2(min_order, seq):  
    return [[y[0], y[1], "met the goal"] if y[1] > min_order  
    else [y[0], y[1], "failed the goal"]  
    for y in [[x[0], x[2] * x[3]]  
    for x in seq]]  
  
print(fun2(200, orders))
```

Generator

```
def fun8(seq, min_order):  
    return ([y[0], y[1], "met the goal"] if y[1] > min_order  
            else [y[0], y[1], "failed the goal"]  
            for y in ([x[0], x[2] * x[3]]  
                      for x in seq))
```

```
> itr = fun8(orders, 200)  
>  
> print(next(itr))  
>  
['34587', 163.8, 'failed the goal']  
> print(next(itr))  
>  
['98762', 284.0, 'met the goal']  
> print(next(itr))  
>  
['77226', 98.85000000000001, 'failed the goal']  
> print(next(itr))  
>  
['88112', 74.97, 'failed the goal']  
> print(next(itr))  
>  
Traceback (most recent call last):  
  File "python", line 1, in <module>  
StopIteration  
> █
```

Summary

- Functional programming allows you to describe the problem in a more abstract way
- Learning functional approach widens your perspective on programming
- It's worth applying when it makes sense
- Python has some useful functional features
- Python 3 is getting more lazy

Functional Programming

Final reflection

Introduction

The design of the **imperative languages** is based directly on the **von Neumann architecture**

Efficiency is the primary concern, rather than the suitability of the language for software development

The design of the **functional languages** is based on **mathematical functions**

A solid theoretical basis that is closer to the user, but relatively unconcerned with the architecture of the machines on which programs will run

Some FP concepts

- A number of interesting programming language concepts have arisen, including:
 - Curried functions
 - Type inferencing
 - Polymorphism
 - Higher-order functions
 - Referential Transparency
 - Lazy evaluation

Curried Functions

- The logician Frege noted in 1883 that we only need functions of one argument.
 - Currying is when you break down a function that takes multiple arguments into a series of functions that take part of the arguments.

```
def add_2(a, b):  
    return a + b
```

```
print(add_2(3,4))
```

```
def add_1(a):  
    def another(b):  
        return a + b  
    return another
```

```
print((add_1(3)(4)))
```

```
def add_1(a):  
    return lambda b: a + b
```

```
print((add_1(3)(4)))
```

Type Inferencing

- Def: ability of the language to infer types without having programmer provide type signatures.
- SML ex: `fun min(a:real,b) = if a > b then b else a`
 - type of `a` has to be given, but then that's sufficient to figure out the type of `b` and the type of `min`
 - What if type of `a` is not specified? Could be `ints` or `bools` or ...
- Overuse of type inferencing in both languages is discouraged
 - declarations are a design aid
 - declarations are a documentation aid
 - declarations are a debugging aid

Higher Order Functions

- Definitions:
 - *zero-order functions*: data in the traditional sense.
 - *first-order functions*: functions that operate on zero-order functions.
 - *second-order functions*: operate on first order
- In general, higher-order functions (HOFs) are those that can operate on functions of any order as long as types match.
 - HOFs are usually polymorphic

Referential Transparency

- *Referential Transparency* - In an FPL, the evaluation of a function always produces the same result given the same parameters

Lazy evaluation

- An evaluation strategy in which arguments to a function are **evaluated only when needed for the computation**.
- Supported by many FPLs including Scheme, Haskell and Common Lisp.
- Usually implemented using **closures** – data structures containing all the information required to evaluate the expression.
- Its opposite, **eager evaluation**, is the usual default in a programming language in which arguments to a function are always evaluated before the function is applied.

Summary: ILs vs FPLs

Imperative Languages:

- Efficient execution
- Complex semantics
- Complex syntax
- Concurrency is programmer designed

Functional Languages:

- Simple semantics
- Simple syntax
- Inefficient execution
- Programs can automatically be made concurrent

- FPLs provide a paradigm for parallel computing
 - absence of assignment (or single assignment) } provide basis
 - independence of evaluation order } for parallel
 - ability to operate on entire data structures } functional programming